

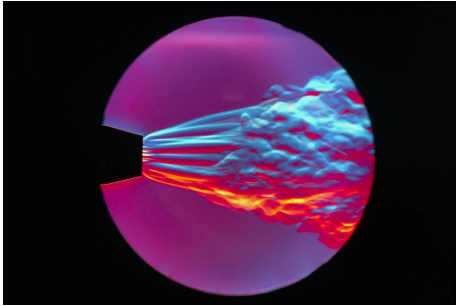


WHAT IS LEAN?

A SHORT PRACTICAL INTRODUCTION

Maths and Lean make the news

OpenAI Says It Has Cracked One of Math's 'Millennium Problems'



'Through this process, A.I. systems can learn to prove mathematical theorems using a computer programming language called *Lean*. The language was originally designed as a tool for human mathematicians. But now that A.I. systems are skillful enough to generate their own computer code, they, too, can use Lean to generate their own math proofs.' (New York Times, September 8th 2026)

Historical development of proof assistants



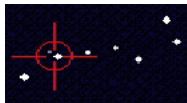
Nicolaas de Bruijn: Automath (L. S. van Benthem Jutting translated Landau's *Foundations of Analysis*)



CompCert: Xavier Leroy et al.: *"Formal verification of the C-compiler."* (2005–)

Proof assistants

Can we prove theorems with a computer instead of just doing difficult calculations?



Many different proof assistants: Rocq, Isabelle/HOL, Lean, Mizar, Agda, ...

- Georges Gonthier: Four colour theorem (Rocq) (2005)
- Georges Gonthier et al.: Odd order theorem (Rocq) (2012)
- Thomas Hales et al.: Kepler conjecture (Isabelle/HOL) (2014)
- Johan Commelin et al. Liquid tensor experiment (Lean) (2023)
- Terence Tao, Yaël Dillies, Bhavik Mehta: Polynomial Freiman–Ruzsa (Lean) (2023)
- Sky Wilshaw: Consistency of New Foundations (Lean) (2024)

What does a proof assistant do?

It checks a mathematical statement down to the foundations.



In Lean's case, the foundations are 'Calculus of inductive constructions' by Thierry Coquand (University of Gothenburg), building on Martin-Löf type theory (Stockholm)

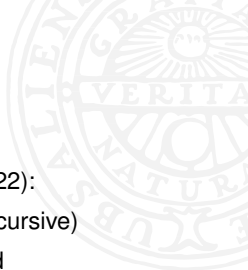


This logic is equiconsistent with ZFC + countably many universes



A small example

```
example {m n : ℕ} (coprime_mn : m.Coprime n) : m ^ 2 ≠ 2 * n ^ 2 := by
  intro sqr_eq
  have h2m : 2 ∣ m := by
    | apply even_of_even_sqr
    | use n ^ 2
  obtain (k, meq) := dvd_iff_exists_eq_mul_left.mp h2m
  have : 2 * (2 * k ^ 2) = 2 * n ^ 2 := by
    | rw [← sqr_eq, meq]
    | ring
  have : 2 * k ^ 2 = n ^ 2 := by
    | apply (mul_right_inj' (by norm_num)).mp this
  have : 2 ∣ n := by
    | apply even_of_even_sqr
    | use k ^ 2
    | exact this.symm
  have : 2 ∣ m.gcd n := by
    | exact Nat.dvd_gcd h2m this
  have : 2 ∣ 1 := by
```



mathlib et al.

- Lean started in 2013 (Leonardo de Moura), Lean 3 in 2017, Lean 4 in 2023
- 2017 mathlib by Mario Carneiro (Chalmers) and Johannes Hölzl (Apple)
- now over 2,000,000 lines of code
- Code is heavily reviewed, 'Bourbaki-style'
- TauCeti (2026), a fast growing human-instructed AI-written library
- Palomar (2026), an online registry for formalised mathematics

My own small contributions (2021/22):

- Catalan numbers (explicit=recursive)
- Kleisli category of a comonad
- Exponent of a group
- generalised order of an element to infinite groups
- Chebyshev polynomials of the 2nd kind
- Generalised quaternion groups

Also some teaching at Uppsala

- Summer Maths/IT Camp (together with Justin Pearson)
- Logic and proof techniques I

Formalisation vs Autoformalisation

- Know that the theorem is true.
 - Understand the argument in all detail.
 - Find the right level of abstraction.
 - Adapt/generalise arguments faster.
- Know that the theorem is true much faster
 - without necessarily needing expert knowledge.



What to check in Lean output?

Semantic alignment problem

- Is the stated theorem what I want to state?
- Are the stated definitions what I want to state?
- Were any non-standard axioms used?
- Do they make use of 'junk values'?
- Do I want a verification of just the theorem or also the proof?

More technical things

- 'native_decide' trusts the compiler, not just the kernel
- Possibility of kernel bugs

```
101 attribute [-simp] GoodReductionJacobian.prodSndPt_prodPairPt GoodReductionJacobian.RelativeGroupLaw.prod
102 attribute [-simp] ModularCurve.JHeronObjectAtP.LevelData.mk.sizeOf_spec ModularCurve.JHeronObjectAtP.L
103 attribute [-simp] ModularCurve.FullLevel.Jac.eval_nsmul ModularCurve.FullLevel.Jac.eval_sub ModularCurve
104 attribute [-simp] ModularCurve.LevelComponent.Raw.mk.injEq ModularCurve.LevelComponent.Raw.mk.sizeOf_spe
105 attribute [-simp] MvFormalGroup.End.toPowerSeries_one MvFormalGroup.End.toPowerSeries_zero MvFormalGroup
106 attribute [-simp] AlgebraicGeometry.SubalgebraStages.specCone_n_app AlgebraicGeometry.SubalgebraStages.s
107 attribute [-simp] ModularCurve.ProjectiveLine.vecMulRow_fst ModularCurve.ProjectiveLine.vecMul_mk HeckeE
108 attribute [-simp] HeckeEis.binaryFormRep_apply_coe ModularCurve.JZeroGoodReductionSpecialization_alt.mk
109 attribute [-simp] GoodReductionJacobian.RelativeGroupLaw.kerPairPointEquiv_apply_coe_coe GoodReductionJa
110 attribute [-simp] IharaLemma.IdempotentSplitting.mk.sizeOf_spec IharaLemma.IdempotentSplitting.coe_toCor
111 attribute [-simp] CuspForm.TWLevel.diamondLin_apply CuspForm.TWLevel.Gen.U.sizeOf_spec CuspForm.TWLevel
112 attribute [-simp] Deformation.TruncWitt.coeff_map Deformation.coe_wittHomVerschiebung Deformation.coe_wi
113 attribute [-simp] TwoChartCech.Cover.kaehlerTrace_mk TwoChartCech.Sections.cupProduct_apply_mk TwoChart
114 attribute [-simp] groupCohomology.totalPairing_apply TaylorWiles.CyclotomicLevelStructure.mk.sizeOf_spe
115 attribute [-simp] Rep.augShortComplex_Xs_Rep.cocycleTwist_single Rep.augShortComplex_Xs_Rep.splittingMod
116 attribute [-simp] Deformation.WittGroup.antipode_X Deformation.WittGroup.comul_X Deformation.WittGroup.p
117 attribute [-simp] CerednikDrinfeld.ShimuraCurveModel.coe_torsionGal_apply CerednikDrinfeld.ShimuraCurveM
118 attribute [-simp] CerednikDrinfeld.ShimuraCurveModel.ModuliWitness.mk.sizeOf_spec CerednikDrinfeld.QM.Fa
119 attribute [-simp] GoodReductionJacobian.BareDeformation.mk.injEq AlgebraicGeometry.SmallExtension.pairFs
120 attribute [-simp] AlgebraicGeometry.RiemannForm.modulePair_fiber AlgebraicGeometry.RiemannForm.pt_LevelL
121 attribute [-simp] AlgebraicGeometry.ThetaLevel.Heis.cen_h AlgebraicGeometry.ThetaLevel.Heis.eta_h Algebr
122 attribute [-simp] MvFormalGroup.CartierModule.mapLinear_apply MvFormalGroup.WittLaw.cVec_add MvFormalGro
123 attribute [-simp] MvFormalGroup.SeriesPoint.val_zero MvFormalGroup.SeriesPoint.val_add MvFormalGroup.Car
124 attribute [-simp] QuaternionAlgebra.qmPeriodMap_apply WeierstrassCurve.frobPoint_zero WeierstrassCurve.f
125 attribute [-simp] CerednikDrinfeld.FormalODModule.gradedSubmodule_toAddSubgroup CerednikDrinfeld.Standar
126 attribute [-simp] FLT.ModelTransfer.ClearedData.mk.injEq FLT.ModelTransfer.reducedChange_u FLT.ModelTran
127
128 theorem fermat_last_theorem (n : ℕ) (hn : 3 ≤ n) (a b c : ℕ) (ha : 0 < a) (hb : 0 < b) (hc : 0 < c) : a
```

Recommended tutorials

- Natural number game
- Mathematics in Lean
- The mechanics of proof